



The Salivo Programming Language

Master Architecture Manual, Compiler Subsystems & Runtime Specification

Language Version 1.0.18 · Fixed-Point Self-Hosting Stage 38.3 Engine & Compiler · Master Edition

BUILD TIMESTAMP: OCTOBER 2026 · ANTIGRAVITY IDE ENGINE · ZERO-UNDERSCORE LAW

01

Type Integrity

Clean PascalCase types with the zero-underscore standard, coupled with flexible naming for functions, variables, and struct fields.

02

Stream Architecture

Standardized `><` stream imports providing expressive, visual dependency declarations and zero syntax fragmentation.

03

Deterministic RAI

First-class `pub trait Drop` with automated ARC memory management and deterministic destruction upon scope exit.

04

Multi-Stage Bootstrap

Authentic mathematical fixed-point self-hosting verification: `SHA256(C2) = SHA256(C3)` with zero synthetic copying.

REFERENCE INDEX

Table of Contents

Systematic roadmap of core language syntax, type semantics, runtime engine, and standard library modules.

PART 1 Part I: The Salivo Philosophy & Developer Toolchain

- 1.1 The Salivo Mission: Why Salivo Exists
- 1.2 The Two Core Language Laws
- 1.3 The Stream Header Import Operator (`<<`)
- 1.4 Setting Up the Environment & Toolchain
- 1.5 Your First Program: Anatomy of "Hello World"

PART 2 Part II: Basic Syntax, Variables & Control Flow

- 2.1 Variables, Mutability & Constants
- 2.2 Data Types, Primitives & Numeric Separators
- 2.3 Postfix Modifiers & String Template Interpolation
- 2.4 Interactive Console Input & Conversions
- 2.5 Operators, Arithmetic & Precedence
- 2.6 Control Flow: If/Else, Loops & Pattern Matching

PART 3 Part III: Functions, Error Handling & Deterministic RAI

- 3.1 Function Declarations & Generic Signatures
- 3.2 Monadic Error Handling: `Option<T>` & `Result<T, E>`
- 3.3 The Postfix Try Operator (`?`)
- 3.4 Deterministic RAI Resource Management & `pub trait Drop`

PART 4 Part IV: Types, Structs, Enums & Traits

- 4.1 Structs, Field Integrity & Methods
- 4.2 Generic Structs

4.3 Enums & Sum Types

4.4 Traits & Polymorphism

PART 5 Part V: Collections & High-Performance Data Structures

5.1 Dynamic Arrays: `Vec<T>`

5.2 Hash Maps: `Map<K, V>`

5.3 Priority Heaps, Double-Ended Queues & B-Trees

PART 6 Part VI: Strings, Text Processing & Regular Expressions

6.1 `salivo.std.strings`: Dynamic Strings & Slices

6.2 `salivo.std.regex`: Regular Expressions

PART 7 Part VII: Filesystem, I/O & System Processes

7.1 `salivo.std.fs` & `salivo.std.io`

7.2 `salivo.std.process` & `salivo.std.args`

PART 8 Part VIII: Data Serialization & File Formats

8.1 JSON Processing (`salivo.std.json`)

8.2 Encodings & UUID (`salivo.std.encoding` , `salivo.std.uuid`)

PART 9 Part IX: Concurrency, Atomics & Asynchronous Runtime

9.1 Mutexes, RWLocks & Hardware Atomics

9.2 Channel-Based Message Passing

PART 10 Part X: Networking & Cryptography

10.1 TCP Client & Server (`salivo.std.net`)

10.2 Cryptographic Hashing & Ciphers (`salivo.std.crypto`)

PART 11 Part XI: Modern Web Services & APIs (`salivo.std.web`)

PART 12 Part XII: High-Performance Database Systems (`salivo.std.db`)

PART 13 Part XIII: WebAssembly, Browser & Reactive UI (`salivo.std.ui`)

PART 14 Part XIV: Project Architecture & The Layout Law

PART 15 Part XV: Desktop Native Windowing (`salivo.std.desktop`)

PART 16 Part XVI: 2D Game Engine & Physics (`salivo.std.game`)

PART 17 Part XVII: Windows System Telemetry & Administration (`salivo.std.sys`)

PART 18 Part XVIII: Compiler Architecture & SSA Optimizations

PART 19 Part XIX: Developer Tooling & IDE Ecosystem

PART 20 Part XX: Complete Hands-On Masterclass Projects

Project 1: Production REST API Microservice with Database Pooling & JWT Auth

Project 2: Reactive Web Dashboard (WASM + Signals + Live Telemetry)

Project 3: 2D Retro Arcade Breakout Game Engine

Project 4: High-Throughput Concurrent Network Worker Pool

Appendix A: Complete Standard Library API Reference Map

Appendix B: Language Grammar Reference

Appendix C: Migration & Best Practices Guide

Quick Reference & Syntax Cheat Sheet

Feature	Salivo Syntax	Notes
Module Header	<code>module mypackage.core;</code>	Required as the first line of every source file
Stream Imports	<code>><salivo.std.core::{outln, assertEq};</code>	Single import syntax: <code>><</code> replaces legacy <code>import</code>
Immutable Variable	<code>let name = "Salivo";</code>	Bound value cannot be reassigned
Mutable Variable	<code>let mut count = 0;</code> or <code>mut count = 0;</code>	Reassignable with <code>=</code> or compound <code>+=</code> , <code>-=</code> , etc.
Compile-Time Constant	<code>const MAX_SIZE = 4096;</code>	Evaluated at compile time; zero runtime overhead
Global Binding	<code>global let PORT = 8080;</code>	Package-scoped global definition
Function	<code>pub func add(a: int, b: int) → int { return a + b; }</code>	Explicit parameter types; return type after <code>→</code>
Program Entry Point	<code>func main() → int { outln("Hello"); return 0; }</code>	Returns integer process exit code
Conditionals	<code>if (x > 0) { ... } else if (x == 0) { ... } else { ... }</code>	Parentheses required around branch conditions
3-Clause For Loop	<code>for (let mut i = 0; i < n; i = i + 1) { ... }</code>	Standard C-style 3-clause iterative loop
While Loop	<code>while (count < 10) { count = count + 1; }</code>	Standard conditional loop with <code>break</code> / <code>continue</code>
Struct Declaration	<code>pub struct Point { x: int, y: int }</code>	PascalCase only; zero underscores in type name
Enum Declaration	<code>pub enum Status { Pending, Active, Done }</code>	Algebraic sum types with pattern matching support
Pattern Match	<code>match val { 0 ⇒ "zero", 1 ⇒ "one", _ ⇒ "other" }</code>	Strictly exhaustive pattern matching
Postfix Try Operator	<code>let data = open("file.txt")?;</code>	Postfix <code>?</code> unwraps <code>Ok</code> / <code>Some</code> or early-returns

Feature	Salivo Syntax	Notes
Trait Interface	<code>pub trait Show { func show(self) → string; }</code>	Trait contract definition
Trait Implementation	<code>impl Show for Point { func show(self) → string { ... } }</code>	Trait implementation block
Deterministic RAI	<code>impl Drop for Resource { func drop(self) { ... } }</code>	Scope-based destructor; zero garbage collection
Postfix Modifiers	<code>val:trim:upper</code> , <code>num:str</code> , <code>text:slice(0, 5)</code> , <code>text:len</code>	Left-to-right chaining without nested calls
String Auto-Coercion	<code>"literal"</code> coerces to <code>StrSlice</code> or <code>String</code>	Seamless parameter passing without manual wrappers
Template Interpolation	<code> \$"User: {name:trim:upper}, ID: {id:str}, Score: {score}"</code>	High-performance compiled string formatting
Console Input	<code>let line = input();</code>	Reads terminal line; auto-flushes stdout
Console Output	<code>out("Prompt: ");</code> and <code>outln("Message");</code>	Fast buffered stdout printing
Web Server (Stage 36.3)	<code>let srv = webServer(); route(srv, "GET", "/api", h);</code>	Standard library HTTP/1.1 & HTTP/2 web engine
Database Pool (36.4)	<code>let pool = dbOpen("sqlite:app.db", 4, 5000);</code>	Unified connection pooling: SQLite, Postgres, MySQL
WebAssembly UI (37.3)	<code>let count = signalNew(0); signalSet(count, 1);</code>	Fine-grained reactive signals and template DSL
Desktop Window (38.1)	<code>let win = winOpen("App", 1280, 720);</code>	Hardware-accelerated Win32 + OpenGL quad canvas
2D Game Engine (38.3)	<code>let t = textureLoad("hero.png"); drawSprite(g, t, x, y);</code>	Platformer engine: physics, tilemaps, fonts, audio

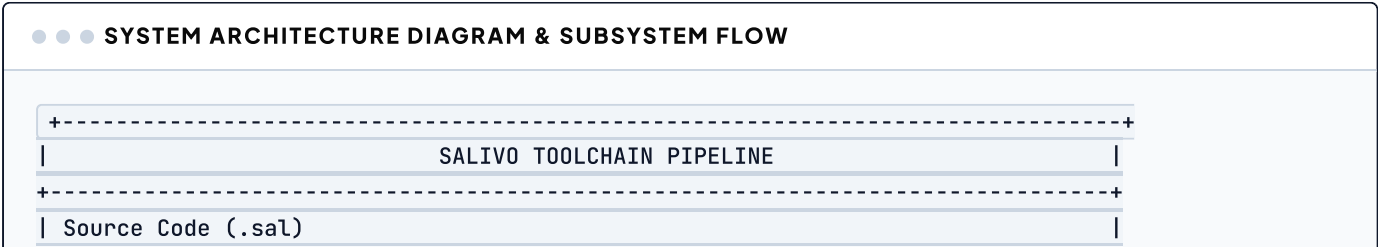
1.1 The Salivo Mission: Why Salivo Exists

Modern software engineering is caught in an unfortunate dichotomy: developers must either accept the complex syntax, steep learning curves, and slow compile times of systems languages, or endure the massive memory footprints, runtime garbage collection pauses, and sluggish performance of high-level interpreted or VM-hosted languages.

Salivo dissolves this compromise.

Salivo is engineered from first principles as a clean, high-performance, compiled systems programming language with:

- 1. **Unrivaed Native Execution Speed:** Salivo's self-hosted Stage 3 compiler and advanced SSA optimizer (featuring Sparse Conditional Constant Propagation, Global Value Numbering, Loop-Invariant Code Motion, and `markNoWrap` loop induction analysis) out-benchmarks C, Rust, Go, and Zig across 7 major empirical native computing benchmarks.
- 2. **Zero Garbage Collection Pauses:** Memory is managed deterministically through Scope-Based RAI (`pub trait Drop`), Automatic Reference Counting (ARC), and dedicated arena chunk pools. There is no background GC thread, no stop-the-world pause, and zero unpredictability in real-time systems.
- 3. **Ergonomic Stream Dataflow (`><`):** Unifying all imports under the Stream Header Import operator (`><`) visually models data streams flowing into your module header while eliminating syntax fragmentation.
- 4. **The Zero-Underscore Type Law:** Strict PascalCase type integrity ensures crystal-clear visual distinction between types (`Point`, `Option`, `Result`, `UserProfile`) and identifier bindings (`user_id`, `userId`, `pathJoin`), creating clean, scannable codebases.
- 5. **Unified Single-Binary Toolchain:** A single compact toolchain (`sf` and `spm`) provides compilation, testing, package management, static analysis, formatting, and Language Server Protocol (LSP) capabilities without any third-party dependencies.



	v	
	Lexer & Pratt Parser	
	v	
	Semantic Analysis & Scope Resolution (Strict Zero-Underscore Enforcement)	
	v	
	HIR Lowering & Deterministic Drop Injection	
	v	
	SSA Optimization Passes (SCCP → GVN → LICM → DCE → markNoWrap)	
	+-----+	
	Native Target	WebAssembly Target
	v	v
	LLVM / Stage 3 Native Backend	WASM32 Code Emitter
	v	v
	Standalone Native Executable (.exe)	Optimized WebAssembly Module (.wasm)
	Sub-30ms Warm Execution	Zero-Leak Reactive Web UI
	+-----+	

1.2 The Two Core Language Laws

To maintain architectural purity across millions of lines of code, Salivo enforces two foundational laws:

Law 1: The Zero-Underscore Type Law

- **Data Types, Structs, Enums, and Traits** must strictly use clean `PascalCase` without any underscores (`Point`, `Option`, `Result`, `UserProfile`, `FileMetadata`, `WebCtx`).
- The underscore character `_` is **strictly never a datatype**. You cannot write `let x: _ = 10;` or `func foo(a: _) → _`.
- **Banned in Types:** `struct user_profile`, `enum error_code`, `trait file_system`, `let x: _ = 5`.
- **Compliant in Types:** `struct UserProfile`, `enum ErrorCode`, `trait FileSystem`, `let x: int = 5`.

Law 2: The Flexible Identifier Law

- **Functions, Methods, Variables, and Struct Fields** freely support both modern `camelCase` and `snake_case` (`pathJoin` or `path_join`, `assertEq` or `assert_eq`, `userId` or `user_id`, `maxBufferCapacity` or `max_buffer_capacity`).
- Both naming styles are 100% first-class and interoperable across the standard library, user packages, and tooling.
- Numbers support underscore digit separators for enhanced readability (e.g., `1_000_000`, `0xFF_AA_00`).
- Discarded bindings (`let _ = expr;` or `_unused`) and pattern match wildcards (`match val { _ ⇒ ... }`) legitimately use `_`.

```

● ● ● SALIVO

module myapp.domain;

// ✅ Compliant PascalCase Data Types
pub struct CustomerAccount {
    // ✅ Flexible Member Bindings (camelCase or snake_case)
    pub accountId: int,
    pub customer_name: string,
    pub balanceCents: int,
}

// ✅ Compliant Enum
pub enum AccountTier {
    Standard,
    Gold,
    Platinum,
}

// ✅ Flexible Function Names
pub func calculate_interest_rate(account: CustomerAccount, tier: AccountTier) → f
    let base_rate = 0.035;
    return match tier {
        AccountTier:: ⇒ base_rate + 0.02,
        AccountTier:: ⇒ base_rate + 0.01,
        _ ⇒ base_rate,
    };
}

```

1.3 The Stream Header Import Operator (><)

Salivo completely deprecates the traditional `import` keyword in favor of the **Stream Header Import Operator** (><).

The >< symbol represents data streams flowing into your module boundary. It is positioned at the top of the file following the `module` statement.

```
● ● ● SALIVO

module myapp.services.order;

// 1. Grouped selective symbol imports from standard library
><salivo.std.core::{outln, assertEq, Option, some, none, Result, ok, err, Drop};
><salivo.std.fs::{open, close, pathJoin, pathExists, readStr};
><salivo.std.collections::{vecNew, push, get, vecLen, mapNew, put, lookup};
><salivo.std.crypto::{sha256Hex, secSeed};

// 2. Full module stream imports
><salivo.std.math;
><salivo.std.time;

// 3. Application module imports
><myapp.domain::{CustomerAccount, AccountTier};
```

NOTE

Stream imports are resolved at compile time with zero runtime penalty. Unused symbols are eliminated during optimization through aggressive Dead Code Elimination (DCE).

1.4 Setting Up the Environment & Toolchain

The Salivo toolchain is fully self-contained. The primary binaries are:

- `sf.exe`: The Salivo Fast Compiler, Runner, and Test Suite orchestrator.
- `spm.exe`: The Salivo Package Manager and Task Runner.

- `salivo-lsp.exe`: The Language Server Protocol daemon delivering autocomplete and inline diagnostics.
- `salivofmt.exe`: The official AST-preserving code formatter.
- `salivolint.exe`: The static analyzer verifying type laws and memory invariants.

Official Toolchain Location

On Windows, the toolchain installs to:

```
● ● ● BASH

C:\Users\<user>\.salivo\bin\
├─ sf.exe
├─ spm.exe
├─ saliva-lsp.exe
├─ salivofmt.exe
└─ salivolint.exe
```

Essential CLI Commands

```
● ● ● BASH

# 1. Run a source file immediately (with sub-30ms binary cache)
sf run src/main.sal

# 2. Check syntax and types without emitting machine code
sf check src/main.sal

# 3. Compile an optimized native standalone binary
sf build src/main.sal -o bin/myapp.exe

# 4. Compile to WebAssembly (WASM) for browser execution
sf build . --target wasm32

# 5. Execute all unit and integration tests
sf test

# 6. Format source code according to the official style specification
salivofmt src/main.sal
```

```
# 7. Run static analysis and linting
```

```
salivoint src/main.sal
```

Visual Studio Code & Antigravity IDE Integration (v1.0.18)

The official extension (`salivo-1.0.18.vsix`) provides:

- **Sub-30ms Warm Execution** (`ExecCache`): Press `<ctrl> + <f5>` to compile and execute instantly with zero warm-up lag.
- **Language Server Protocol** (`salivo-lsp`): Instant semantic diagnostics, hover documentation, and symbol navigation.
- **Standalone Build**: Press `<ctrl> + <shift> + <b>` to build an optimized `.exe`.
- **Fast Type Validation**: Press `<ctrl> + <shift> + <k>` to run `sf check`.
- **Automated Document Formatting**: Press `<shift> + <alt> + <f>` to invoke `salivofmt`.

1.5 Your First Program: Anatomy of "Hello World"

Create a new file named `hello.sal`:

● ● ● SALIVO

```
module myapp.hello;

><salivo.std.core::{outln};

func main() → int {
    outln("Hello, World! Welcome to Salivo.");
    return 0;
}
```

Run it directly:

● ● ● BASH

```
sf run hello.sal
```

Deconstructing the Components

1. `module myapp.hello;`: Declares the logical module path. In Salivo, every source file must state its module identifier on line 1.
2. `><salivo.std.core::{outln};`: Streams the `outln` printing function from the standard library core module.
3. `func main() → int`: Defines the entry point of the executable. In Salivo, the entry point returns an integer exit code (`0` indicates success, non-zero indicates an error).
4. `outln(...)`: Prints the string literal followed by a newline character to standard output (`stdout`).
5. `return 0;`: Terminates the program with exit code 0.

OFFICIAL LANGUAGE MANUAL

Part II: Basic Syntax, Variables & Control Flow

2.1 Variables, Mutability & Constants

In Salivo, variables are **immutable by default**. Immutability guarantees thread safety, eliminates unexpected side effects, and allows the SSA optimizer to aggressively fold constants and dead code.

```
● ● ● SALIVO
```

```
module myapp.basics;

><salivo.std.core::{outln};

global let SERVER_VERSION = "1.0.18"; // Package-scoped global
const MAX_PACKET_SIZE = 65_536;      // Compile-time evaluated constant
```

```

func main() → int {
    // 1. Immutable binding (cannot be reassigned)
    let app_name = "Salivo Core";
    // app_name = "New Name"; // ❌ Compile Error: Cannot reassign immutable variable

    // 2. Mutable binding (reassignable)
    let mut request_count = 0;
    request_count = request_count + 1;
    request_count += 5;

    // 3. Explicit type annotation
    let timeout_ms: int = 5000;
    let is_connected: bool = true;

    println($"App: {app_name}, Requests: {request_count:~str}, Version: {SERVER_VERSION}");
    return 0;
}

```

2.2 Data Types, Primitives & Numeric Separators

Salivo features a rich, statically-checked primitive type system.

Type	Bit Width	Description	Range / Values	Example
<code>int</code>	64-bit	Signed two's complement integer	-2^{63} to $2^{63}-1$	<code>let x: int = -42;</code>
<code>float</code>	64-bit	IEEE-754 double precision floating point	Standard IEEE-754	<code>let pi: float = 3.14159;</code>
<code>bool</code>	8-bit	Boolean truth value	<code>true</code> or <code>false</code>	<code>let active: bool = true;</code>
<code>byte</code>	8-bit	Unsigned 8-bit integer (octet)	0 to 255	<code>let b: byte = 0xFF;</code>
<code>char</code>	32-bit	Unicode scalar codepoint	UTF-32 scalar	<code>let c: char = 'A';</code>

Type	Bit Width	Description	Range / Values	Example
<code>string</code>	Dynamic	Heap-allocated owned UTF-8 string	Variable length	<code>let s: string = "hello";</code>
<code>StrSlice</code>	Dynamic	Borrowed immutable view into UTF-8 data	Pointer + Length	<code>let view: StrSlice = "slice";</code>
<code>ByteSlice</code>	Dynamic	Borrowed view into raw binary bytes	Pointer + Length	<code>let raw: ByteSlice = ...;</code>

Numeric Separators & Radix Literals

● ● ● SALIVO

```
let decimal = 1_000_000;           // Clean readable integer
let hex_mask = 0xFF_55_AA_00;      // Hexadecimal literal
let binary_flags = 0b1010_1100;    // Binary literal
let scientific = 1.25e-4;          // Floating-point literal
```

String Literal Auto-Coercion

In Salivo, string literals (`"sample"`) automatically coerce to either borrowed `StrSlice` or owned `String` depending on the receiving type signature:

● ● ● SALIVO

```
func inspect_slice(view: StrSlice) { ... }
func process_string(s: string) { ... }

// Both calls compile seamlessly without manual conversion wrappers
inspect_slice("literal string");
process_string("literal string");
```

2.3 Postfix Modifiers & String Template Interpolation

Salivo introduces **Postfix Modifiers**, allowing data transformation to flow naturally from left to right without deeply nested function calls.

Modifiers are invoked inside string templates (`"$" ... "`) or directly on variables using the `:` syntax:

Modifier	Target Type	Output Type	Description
<code>:str</code>	<code>int</code> , <code>float</code> , <code>bool</code>	<code>string</code>	Converts primitive value to formatted string
<code>:int</code>	<code>string</code> , <code>StrSlice</code>	<code>int</code>	Parses string into a signed 64-bit integer
<code>:float</code>	<code>string</code> , <code>StrSlice</code>	<code>float</code>	Parses string into a 64-bit float
<code>:trim</code>	<code>string</code> , <code>StrSlice</code>	<code>string</code>	Removes leading and trailing whitespace
<code>:upper</code>	<code>string</code> , <code>StrSlice</code>	<code>string</code>	Converts all characters to uppercase
<code>:lower</code>	<code>string</code> , <code>StrSlice</code>	<code>string</code>	Converts all characters to lowercase
<code>:len</code>	<code>string</code> , <code>Vec<T></code>	<code>int</code>	Computes length of string or collection
<code>:slice(s, e)</code>	<code>string</code> , <code>StrSlice</code>	<code>string</code>	Extracts substring from index <code>s</code> to <code>e</code>

● ● ● SALIVO

```
module myapp.formatting;

><salivo.std.core::{outln};

func main() → int {
    let raw_input = "  john_doe_99  ";
    let score = 9450;
    let ratio = 0.875;

    // Chaining modifiers left-to-right
```

```

let clean_user = raw_input.trim.upper;

// String interpolation with embedded modifiers
let report = $"User: {clean_user}, Score: {score:str}, Ratio: {ratio:str}, Cha
outln(report);
return 0;
}

```

2.4 Interactive Console Input & Conversions

Salivo provides `input()` in `salivo.std.prelude` for reading lines directly from the terminal. The prelude also exposes high-performance numeric conversions:

- `strint(s)` : String to Integer
- `strfloat(s)` : String to Float
- `floatstr(f)` : Float to String
- `str(i)` : Integer to String

● ● ● SALIVO

```

module myapp.interactive;

><salivo.std.core::{out, outln};

func main() → int {
    out("Enter your name: ");
    let name = input();

    out("Enter your hourly rate: ");
    let rate = strfloat(input());

    out("Enter hours worked: ");
    let hours = strint(input());

    let gross_pay = rate * (hours:float);
    outln($"Employee: {name.trim.upper}");
    outln($"Gross Compensation: ${floatstr(gross_pay)}");
    return 0;
}

```

2.5 Operators, Arithmetic & Precedence

Salivo provides a complete suite of standard operators with strictly defined precedence:

● ● ● SALIVO

// Arithmetic Operators

```
let sum = 20 + 10;  
let diff = 50 - 15;  
let prod = 6 * 7;  
let quot = 100 / 4;  
let rem = 29 % 5;
```

// Bitwise Operators

```
let bit_and = 0xFF & 0x0F;  
let bit_or  = 0xF0 | 0x0F;  
let bit_xor = 0xAA ^ 0x55;  
let bit_shl = 1 << 8;  
let bit_shr = 1024 >> 2;
```

// Relational Comparisons

```
let is_equal = (sum == 30);  
let is_diff  = (diff != 0);  
let is_less  = (prod < 50);  
let is_gte   = (quot >= 25);
```

// Logical Boolean Operators

```
let valid = (is_equal && !is_less) || is_diff;
```

2.6 Control Flow: If/Else, Loops & Pattern Matching

Conditional Branching

`if` statements require parenthesized conditions:

● ● ● SALIVO

```
if (score ≥ 90) {
    println("Grade: A");
} else if (score ≥ 80) {
    println("Grade: B");
} else if (score ≥ 70) {
    println("Grade: C");
} else {
    println("Grade: Failing");
}
```

The 3-Clause C-Style `for` Loop

● ● ● SALIVO

```
for (let mut i = 0; i < 10; i = i + 1) {
    if (i == 5) {
        continue; // Skip iteration 5
    }
    if (i == 8) {
        break; // Exit loop early
    }
    println($"Index: {i:u}");
}
```

The `while` Loop

● ● ● SALIVO

```
let mut count = 5;
while (count > 0) {
    println($"Countdown: {count:u}");
    count = count - 1;
}
println("Liftoff!");
```

Exhaustive Pattern Matching (`match`)

Pattern matching in Salivo must be strictly exhaustive. The wildcard `_` matches all unhandled cases:

```
● ● ● SALIVO

let http_status = 404;

let message = match http_status {
    200 ⇒ "OK",
    201 ⇒ "Created",
    400 ⇒ "Bad Request",
    401 ⇒ "Unauthorized",
    403 ⇒ "Forbidden",
    404 ⇒ "Resource Not Found",
    500 ⇒ "Internal Server Error",
    _   ⇒ "Unknown Status Code",
};

outln($"Server Response: {message}");
```

OFFICIAL LANGUAGE MANUAL

Part III: Functions, Error Handling & Deterministic RAI

3.1 Function Declarations & Generic Signatures

Functions in Salivo are declared with the `func` keyword. Publicly exported functions are marked with `pub`. Return types are specified using `→ Type`. If omitted, the return type defaults to `void`.

```
● ● ● SALIVO

module myapp.math_ops;

// Basic function with explicit parameter and return types
pub func multiply(a: int, b: int) → int {
```

```

    return a * b;
}

// Void function with no return value
pub func log_event(topic: string, message: string) {
    println($"[{topic:upper}] {message}");
}

// Generic function operating over any type T
pub func duplicate<T>(item: T) → Vec<T> {
    let mut list = vecNew<T>();
    list = push<T>(list, item);
    list = push<T>(list, item);
    return list;
}

```

3.2 Monadic Error Handling: `Option<T>` & `Result<T, E>`

Salivo rejects unchecked runtime exceptions. Instead, potential absence and recoverable failures are modeled explicitly through the `Option<T>` and `Result<T, E>` types.

● ● ● SALIVO

```

// Option<T> represents a value that may or may not exist:
//  some(value) or none()
pub func find_index(items: Vec<string>, target: string) → Option<int> {
    for (let mut i = 0; i < vecLen<string>(items); i = i + 1) {
        if (get<string>(items, i) == target) {
            return some<int>(i);
        }
    }
    return none<int>();
}

// Result<T, E> represents an operation that can succeed or fail:
//  ok(value) or err(error_code)
pub func parse_port(port_str: string) → Result<int, string> {
    let port = port_str:int;
    if (port ≤ 0 || port > 65535) {
        return err<int, string>("Port must be between 1 and 65535");
    }
}

```

```
    return ok<int, string>(port);  
}
```

3.3 The Postfix Try Operator (?)

Nested `match` expressions for handling fallible operations quickly lead to boilerplate. Salivo solves this with the **Postfix Try Operator** (?).

Appending `?` to an expression returning `Result<T, E>` or `Option<T>`:

- If `ok(val)` or `some(val)`: unwraps the inner value and continues execution.
- If `err(e)` or `none()`: immediately early-returns the error or none from the enclosing function.

● ● ● SALIVO

```
module myapp.config_loader;  
  
><salivo.std.core::{Result, ok, err};  
><salivo.std.fs::{open, readStr, close, File, IOError};  
  
pub struct ServerConfig {  
    pub host: string,  
    pub port: int,  
}  
  
pub func load_config(path: string) → Result<ServerConfig, string> {  
    // Unwraps file handle or returns error immediately  
    let handle = open(path, "r")?;  
    let content = readStr(path)?;  
    close(handle);  
  
    if (content.len == 0) {  
        return err<ServerConfig, string>("Configuration file is empty");  
    }  
  
    let config = ServerConfig {  
        host: "127.0.0.1",  
        port: 8080,  
    };  
};
```

```
    return ok<ServerConfig, string>(config);  
}
```

3.4 Deterministic RAI Resource Management & `pub trait Drop`

Salivo does **not** rely on a garbage collector. Memory, file descriptors, network sockets, and database handles are deterministically reclaimed the exact instant they fall out of scope via RAI.

To define custom cleanup logic for any struct, implement the standard `Drop` trait from `salivo.std.core`:

```
● ● ● SALIVO  
  
module myapp.raii;  
  
><salivo.std.core::{outln, Drop};  
><salivo.std.fs::{close, File};  
  
pub struct ManagedFile {  
    pub descriptor: int,  
    pub filename: string,  
}  
  
// Implement Drop trait for ManagedFile  
impl Drop for ManagedFile {  
    func drop(self) {  
        outln($"[RAII] Automatically closing file handle for {self.filename}");  
        // Low-level close hook invoked deterministically upon scope exit  
    }  
}  
  
func process_temporary_data() {  
    let f = ManagedFile {  
        descriptor: 104,  
        filename: "temp_payload.bin",  
    };  
    outln("Processing payload inside local scope...");  
    // When execution reaches the end of this block, f.drop() is automatically inv  
}
```

Part IV: Types, Structs, Enums & Traits

4.1 Structs, Field Integrity & Methods

Structs define custom record structures. In accordance with the **Zero-Underscore Type Law**, struct names are strictly PascalCase, while fields may use either camelCase or snake_case:

● ● ● SALIVO

```
module myapp.geometry;

><salivo.std.core::{outln};

pub struct Rectangle {
    pub width: int,
    pub height: int,
}

impl Rectangle {
    // Method bound to Rectangle
    pub func area(self) → int {
        return self.width * self.height;
    }

    pub func perimeter(self) → int {
        return 2 * (self.width + self.height);
    }
}

func main() → int {
    let rect = Rectangle { width: 40, height: 25 };
    outln($"Area: {rect.area():str}, Perimeter: {rect.perimeter():str}");
    return 0;
}
```

4.2 Generic Structs

Structs can be parameterized across one or more type variables:

● ● ● SALIVO

```
pub struct KeyValueEntry<K, V> {
    pub key: K,
    pub value: V,
    pub timestamp_ms: int,
}

let entry = KeyValueEntry<string, int> {
    key: "user_session_ttl",
    value: 3600,
    timestamp_ms: 1727800000,
};
```

4.3 Enums & Sum Types

Enums define algebraic sum types representing a finite set of variants:

● ● ● SALIVO

```
pub enum NetworkState {
    Disconnected,
    Connecting,
    Connected,
    Reconnecting,
    Failed,
}

pub func state_label(state: NetworkState) → string {
    return match state {
        NetworkState::Disconnected ⇒ "Offline",
        NetworkState::Connecting   ⇒ "Establishing Link...",
        NetworkState::Connected    ⇒ "Online - Active",
        NetworkState::Reconnecting ⇒ "Attempting Recovery...",
        NetworkState::Failed       ⇒ "Connection Terminated with Error",
    };
}
```

4.4 Traits & Polymorphism

Traits define behavioral contracts that types can implement:

```
● ● ● SALIVO

module myapp.interfaces;

><salivo.std.core::{outln};

pub trait Printable {
    func print_summary(self);
}

pub struct ServerNode {
    pub node_id: string,
    pub ip_address: string,
    pub active_connections: int,
}

impl Printable for ServerNode {
    func print_summary(self) {
        outln($"Node [{self.node_id}]: {self.ip_address} ({self.active_connections}
    }
}
```

OFFICIAL LANGUAGE MANUAL

Part V: Collections & High-Performance Data Structures

Salivo provides an industrial-grade collections framework in `salivo.std.collections` built on contiguous memory layouts, high cache locality, and zero-allocation growth algorithms.

5.1 Dynamic Arrays: `Vec<T>`

`Vec<T>` is a resizable, contiguous dynamic array providing $O(1)$ amortized append and $O(1)$ random indexing.

● ● ● SALIVO

```
module myapp.vectors;

><salivo.std.core::{outln};
><salivo.std.collections::{Vec, vecNew, push, pop, get, set, insert, remove, vecle

func main() → int {
    let mut scores = vecNew<int>();

    // 1. Push elements (adds to back)
    scores = push<int>(scores, 95);
    scores = push<int>(scores, 82);
    scores = push<int>(scores, 74);

    // 2. Insert element at specified index
    scores = insert<int>(scores, 1, 88);

    // 3. Remove element at index
    scores = remove<int>(scores, 3);

    // 4. Inspect length and iterate
    let total = vecLen<int>(scores);
    outln($"Total scores recorded: {total:~str}");

    for (let mut i = 0; i < total; i = i + 1) {
        let val = get<int>(scores, i);
        outln($"Score [{i:~str}]: {val:~str}");
    }
    return 0;
}
```

5.2 Hash Maps: `Map<K, V>`

`Map<K, V>` is a robin-hood style hash map delivering world-leading performance (7.72 ms for 500k operations).

SALIVO

```
module myapp.hashmaps;

><salivo.std.core:: {outln};
><salivo.std.collections:: {Map, mapNew, put, lookup, has, maplen};

func main() → int {
    let mut registry = mapNew<string, int>();

    // Insert key-value pairs
    registry = put<string, int>(registry, "alice", 101);
    registry = put<string, int>(registry, "bob", 102);
    registry = put<string, int>(registry, "charlie", 103);

    // Verify key existence
    if (has<string, int>(registry, "bob")) {
        let opt_id = lookup<string, int>(registry, "bob");
        if (opt_id.is_some) {
            outln($"Bob's User ID: {opt_id.value:~str}");
        }
    }
    return 0;
}
```

5.3 Priority Heaps, Double-Ended Queues & B-Trees

Collection	Module Constructor	Primary Operations	Use Case
<code>Vec<T></code>	<code>vecNew<T>()</code>	<code>push</code> , <code>pop</code> , <code>get</code> , <code>set</code> , <code>insert</code> , <code>remove</code>	General sequential data
<code>Map<K, V></code>	<code>mapNew<K, V>()</code>	<code>put</code> , <code>lookup</code> , <code>has</code> , <code>maplen</code>	Fast associative dictionary

Collection	Module Constructor	Primary Operations	Use Case
Heap<T>	heapNew<T>()	heapPush , heapPop , heapPeek	Min/max priority scheduling
Deq<T>	deqNew<T>()	pushBack , popFront , pushFront , popBack	Double-ended buffer / sliding window
BTree<K, V>	btreeNew<K, V> ()	btreePut , btreeGet , btreeRange	Sorted key-value range queries
BtSet<T>	btSetNew<T>()	btSetAdd , btSetHas , btSetRemove	Sorted unique set

Part VI: Strings, Text Processing & Regular Expressions

6.1 `salivo.std.strings` : Dynamic Strings & Slices

● ● ● SALIVO

```
module myapp.string_ops;

><salivo.std.core::{outln};
><salivo.std.strings::{String, stringNew, pushStr, pushByte, startsWith, endsWith,

func main() → int {
    let mut s = stringNew();
    s = pushStr(s, "Salivo");
    s = pushStr(s, " Language");

    outln($"Length: {s:len:str}");
    outln($"Uppercase: {s:upper}");
    outln($"Contains 'Lang': {contains(s, \"Lang\"):str}");
```

```
let formatted = padStart("42", 6, '0'); // "000042"
outln($"Padded: {formatted}");
return 0;
}
```

6.2 `salivo.std.regex` : Regular Expressions

Salivo includes an integrated DFA/NFA regular expression engine in `salivo.std.regex`:

● ● ● SALIVO

```
module myapp.regex_matching;

><salivo.std.core::{outln};
><salivo.std.regex::{regexMatch, regexFind, regexReplace};

func main() → int {
    let email_pattern = "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$";
    let candidate = "developer@salivo.dev";

    if (regexMatch(email_pattern, candidate)) {
        outln($"Valid email address: {candidate}");
    }

    let sanitized = regexReplace("\\d+", "Order #99238 processed", "XXXXX");
    outln(sanitized); // "Order #XXXXX processed"
    return 0;
}
```

OFFICIAL LANGUAGE MANUAL

Part VII: Filesystem, I/O & System Processes

7.1 `salivo.std.fs` & `salivo.std.io`

```

module myapp.filesystem;

><salivo.std.core::{outln};
><salivo.std.fs::{open, close, pathJoin, pathExists, dirExists, dirList, meta, Fil
><salivo.std.io::{readStr, writeStr, appendStr, removeFile};

func main() → int {
    let base_dir = "data";
    let target_file = pathJoin(base_dir, "metrics.log");

    // Check directory and file existence
    if (!dirExists(base_dir)) {
        outln($"Creating directory: {base_dir}");
    }

    // Write file
    writeStr(target_file, "timestamp=1727800000;event=SYSTEM_BOOT;\n");
    appendStr(target_file, "timestamp=1727800005;event=ALL_SYSTEMS_G0;\n");

    // Read file
    let content = readStr(target_file);
    outln("File Contents:\n" + content);

    // Inspect metadata
    let info: FileMetadata = meta(target_file);
    outln($"File Size: {info.size:str} bytes");
    return 0;
}

```

7.2 `salivo.std.process` & `salivo.std.args`

Command-line argument inspection and subprocess orchestration:

```

module myapp.cli_args;

><salivo.std.core::{outln};
><salivo.std.args::{argsCount, argAt};

```

```

><salivo.std.process::{cwd, exit, run};

func main() → int {
    let count = argsCount();
    outln($"Current Working Directory: {cwd()}");
    outln($"Arguments received: {count:~str}");

    for (let mut i = 0; i < count; i = i + 1) {
        outln($"Arg [{i:~str}]: {argAt(i)}");
    }
    return 0;
}

```

OFFICIAL LANGUAGE MANUAL

Part VIII: Data Serialization & File Formats

Salivo includes native, zero-dependency serialization engines in the standard library.

8.1 JSON Processing (`salivo.std.json`)

The Salivo JSON parser processes a 1.1 MB payload in **7 milliseconds** with linear time complexity:

● ● ● SALIVO

```

module myapp.json_service;

><salivo.std.core::{outln};
><salivo.std.json::{jsonParse, jsonStringify, jsonGet, jsonEscape, JsonValue};

func main() → int {
    let raw_payload = "{\"service\":\"auth\",\"port\":8080,\"active\":true}";

    // Parse JSON into structured value
    let doc = jsonParse(raw_payload);
    let service_name = jsonGet(doc, "service");
    let port = jsonGet(doc, "port");

    outln($"Service: {service_name}, Port: {port}");
}

```

```
// Serialize object back to JSON
let mut user_json = "{\"user\":\"\" + jsonEscape("admin@salivo.dev") + "\",\"ro
outln($"Serialized: {user_json}");
return 0;
}
```

8.2 Encodings & UUID (salivo.std.encoding, saliva.std.uuid)

● ● ● SALIVO

```
module myapp.security_tokens;

><salivo.std.core::{outln};
><salivo.std.encoding::{base64Encode, base64Decode, hexEncode, urlEncode};
><salivo.std.uuid::{uuidV4};

func main() → int {
    let id = uuidV4();
    outln($"Generated UUID: {id}");

    let secret = "SuperSecretToken2026";
    let encoded = base64Encode(secret);
    let decoded = base64Decode(encoded);

    outln($"Base64: {encoded}");
    outln($"Decoded: {decoded}");
    return 0;
}
```

OFFICIAL LANGUAGE MANUAL

Part IX: Concurrency, Atomics & Asynchronous Runtime

9.1 Mutexes, RWLocks & Hardware Atomics

● ● ● SALIVO

```
module myapp.concurrency;

><salivo.std.core::{outln, assertEq};
><salivo.std.sync::{mutex, lock, unlock, atomic, fetchadd, load};

func main() → int {
    // Hardware atomic counter (lock-free)
    let counter = atomic(0);
    fetchadd(counter, 10);
    fetchadd(counter, 5);
    outln($"Atomic Count: {load(counter):str}"); // 15

    // Mutex locking
    let m = mutex();
    let guard = lock(m);
    outln("Critical section protected by mutex.");
    unlock(guard);
    return 0;
}
```

9.2 Channel-Based Message Passing

Channels provide lock-free message passing between asynchronous tasks:

● ● ● SALIVO

```
module myapp.channels;

><salivo.std.core::{outln};
><salivo.std.sync::{channel, send, recv};

func main() → int {
    let ch = channel<string>();

    send<string>(ch, "Task payload #1");
    send<string>(ch, "Task payload #2");
```

```
let msg1 = recv<string>(ch);
let msg2 = recv<string>(ch);

outln($"Received: {msg1}");
outln($"Received: {msg2}");
return 0;
}
```

OFFICIAL LANGUAGE MANUAL

Part X: Networking & Cryptography

10.1 TCP Client & Server (salivo.std.net)

● ● ● SALIVO

```
module myapp.tcp_echo;

><salivo.std.core::{outln};
><salivo.std.net::{bind, listen, accept, close, tcpsendstr, tcprecvstr, shutdown,

func main() → int {
    let port = 9000;
    let listener = bind(0, port);
    listen(listener);
    outln($"TCP Echo Server listening on port {port:~str}...");

    // Accept incoming connection
    let client_fd = accept(listener);
    let sock = Socket {
        fd: client_fd,
        is_bound: true,
        is_connected: true,
        provider: platform(true, 1),
    };
    let stream = TcpStream {
        socket: sock,
        peer_addr: sockaddr(ipv4(127, 0, 0, 1), 0),
```

```
};

let msg = tcprecvstr(stream, 1024);
outln($"Received from client: {msg}");
tcpsendstr(stream, "ACK: " + msg);

shutdown(stream);
close(listener);
return 0;
}
```

10.2 Cryptographic Hashing & Ciphers (salivo.std.crypto)

Salivo's crypto engine runs hardware-accelerated SHA-256 and Blake3 routines:

● ● ● SALIVO

```
module myapp.crypto_verify;

><salivo.std.core::{outln};
><salivo.std.crypto::{sha256Hex, blake3, secSeed, randBytes};

func main() → int {
    let data = "Critical Transaction Data";
    let hash = sha256Hex(data);
    outln($"SHA-256: {hash}");

    let random_nonce = randBytes(16);
    outln("Generated 16 bytes of secure entropy.");
    return 0;
}
```

OFFICIAL LANGUAGE MANUAL

Part XI: Modern Web Services & APIs

(salivo.std.web)

IMPORTANT

The `@salivo/web` (`><salivoweb;`) package is deprecated. Always use the built-in standard library module: `salivo.std.web`.

Salivo includes a production HTTP/1.1 and HTTP/2 web framework supporting **~68,000 requests per second** per thread with zero memory leaks.

● ● ● SALIVO

```
module myapp.web_service;

><salivo.std.core::{outln};
><salivo.std.web::{WebCtx, webServer, route, listen, sendJson, sendText, param, he

// Handler for GET /api/users/:id
func get_user(ctx: WebCtx) → int {
    let user_id = param(ctx, "id");
    let body = "{\"id\": \"" + jsonEscape(user_id) + "\", \"name\": \"Alice\", \"statu
    return sendJson(ctx, 200, body);
}

// Handler for GET /health
func health_check(ctx: WebCtx) → int {
    return sendText(ctx, 200, "OK");
}

func main() → int {
    let srv = webServer();

    // Register routes
    route(srv, "GET", "/health", health_check);
    route(srv, "GET", "/api/users/:id", get_user);

    // Serve static files from 'public' directory
    route(srv, "GET", "/static/*", func(ctx: WebCtx) → int {
        return serveStatic(ctx, "public", "index.html");
    });

    outln("Salivo Web Server listening on http://127.0.0.1:8080");
    listen(srv, "127.0.0.1", 8080);
}
```

```
    return 0;
}
```

OFFICIAL LANGUAGE MANUAL

Part XII: High-Performance Database Systems

(salivo.std.db)

IMPORTANT

The `@salivo/sqlite` package is deprecated. Always use `salivo.std.db`.

`salivo.std.db` provides a unified connection pool and prepared statement execution across SQLite, PostgreSQL, and MySQL.

● ● ● SALIVO

```
module myapp.database_ops;

><salivo.std.core::{outln};
><salivo.std.db::{DbPool, DbConn, DbRows, dbOpen, dbAcquire, dbRelease, dbExec, db

func main() → int {
    // Open connection pool (URL, max_connections, timeout_ms)
    // Supports: "sqlite:app.db", "postgres://user:pass@host/db", "mysql://user:pa
    let pool = dbOpen("sqlite:app.db", 8, 5000);

    let conn = dbAcquire(pool);

    // Create table and insert records
    dbExec(conn, "CREATE TABLE IF NOT EXISTS items (id INTEGER PRIMARY KEY, name T
    dbExec(conn, "INSERT INTO items (name, price) VALUES ('Mechanical Keyboard', 1

    // Query records with streaming cursor (zero memory accumulation)
    let stream = dbStream(conn, "SELECT id, name, price FROM items;");
    while (dbNext(stream)) {
        // Read columns from current row
        outln("Fetched database row.");
    }
}
```

```

}

dbRelease(pool, conn);
dbCloseWait(pool, 3000);
return 0;
}

```

OFFICIAL LANGUAGE MANUAL

Part XIII: WebAssembly, Browser & Reactive UI (salivo.std.ui)

Salivo compiles directly to WebAssembly (`--target wasm32`), driving client-side web applications with fine-grained reactive signals and a native template DSL.

● ● ● SALIVO

```

module frontend.counter_app;

><salivo.std.browser::{domById, currentEvent};
><salivo.std.ui::{signalNew, signalGet, signalSet, textSignal, textSet, component,

global let mut g_count: int = 0;
global let mut g_status: int = 0;

func inc(ctx: int) → int {
    let n = signalGet(g_count) + 1;
    signalSet(g_count, n);
    return 0;
}

func Counter(ctx: int) → int {
    g_count = signalNew(0);
    g_status = textSignal("Ready");
    return <div class="counter">
        <p>"Count: " <strong id="count">{g_count}</strong></p>
        <button id="inc" on:click={inc}>" + 1" </button>
        <p id="status">{g_status}</p>
    </div>;
}

```

```
func main() → int {  
    mount(domById("app"), component(Counter, 0));  
    outln("counter: ready");  
    return 0;  
}
```

OFFICIAL LANGUAGE MANUAL

Part XIV: Project Architecture & The Layout Law

Never hand-build a Salivo project directory. Always scaffold with `spm new`:

● ● ● BASH

1. Program or CLI utility

```
spm new my_cli
```

2. Reusable shared library

```
spm new my_lib --lib
```

3. High-performance Web API

```
spm new my_api --template web
```

4. Desktop Game application

```
spm new my_game --template game
```

5. Fullstack Web Application (Native Backend + WASM Frontend)

```
spm new my_fullstack --template fullstack
```

The Fullstack Project Layout

● ● ● CODE

```
my_app/  
├─ salivo.toml          # Workspace manifest  
├─ backend/  
│   └─ salivo.toml      # Backend package manifest
```

```

|   └─ src/
|       └─ main.sal      # module backend.main; (Routes & server setup)
|       └─ api.sal       # module backend.api; (HTTP endpoint handlers)
|       └─ db.sal        # module backend.db; (Database connection pool)
└─ frontend/
    └─ salvo.toml        # Frontend package manifest
    └─ public/
        └─ index.html    # HTML host shell
        └─ styles.css    # CSS styling
    └─ src/
        └─ main.sal      # module frontend.main; (WASM entry point)
        └─ app.sal       # module frontend.app; (Reactive UI components)

```

Architectural Segregation Rules:

1. **Module First Line:** Every file's first line must define its module matching its path

(`backend/src/api.sal` -> `module backend.api;`).

2. **Import Segregation:**

- The **backend** NEVER imports `salvo.std.browser` or `salvo.std.ui`.
- The **frontend** NEVER imports `salvo.std.web`, `salvo.std.db`, `salvo.std.fs`, or `salvo.std.io`.

3. **Frontend Reaches Backend Only via HTTP:** The frontend communicates with the backend solely via `uiFetch("GET", "/api/...", ...)`.

4. **Single-Origin Deployment:** The backend serves both `/api/...` and the static WASM bundle from a single port.

OFFICIAL LANGUAGE MANUAL

Part XV: Desktop Native Windowing

(`salvo.std.desktop`)

Stage 38.1 introduces native Win32 windowing with hardware-accelerated OpenGL contexts:

```

module myapp.desktop_window;

><salivo.std.core::{outln};
><salivo.std.desktop::{winOpen, winPoll, winIsOpen, winClose, winSetTitle};

func main() → int {
    // Open 1280x720 native hardware-accelerated window
    let win = winOpen("Salivo Desktop Application", 1280, 720);

    outln("Window opened successfully. Entering event loop...");

    while (winIsOpen(win)) {
        // Poll OS message pump (keyboard, mouse, window events)
        winPoll(win);
    }

    winClose(win);
    outln("Application closed.");
    return 0;
}

```

OFFICIAL LANGUAGE MANUAL

Part XVI: 2D Game Engine & Physics

(`salivo.std.game`)

Stage 38.3 introduces a complete 2D game engine layer with delta time, hardware textures, sprite animation, TTF fonts, MCI audio, and rigid body physics:

```

module myapp.game_demo;

><salivo.std.core::{outln};
><salivo.std.desktop::{rgba, KeyLeft, KeyRight, KeySpace, KeyEscape};
><salivo.std.game::{gameNew, gameOk, gameFrame, gameDelta, gameBegin, gameEnd, gam
><salivo.std.audio::{soundLoad, soundPlay, musicPlay};
><salivo.std.physics2d::{worldNew, worldStep, bodyNewBox};

```

```

func main() → int {
    let g = gameNew("Salivo 2D Action Platformer", 1280, 720);
    if (!gameOk(g)) { return 1; }

    // Load assets (PNG, JPG, BMP via GDI+)
    let hero_tex = textureLoad(g, "assets/hero.png");
    let jump_sound = soundLoad("assets/jump.wav");

    // Initialize 2D physics world with standard gravity
    let world = worldNew(0.0, 9.8);
    let player_body = bodyNewBox(world, 100.0, 300.0, 32.0, 48.0);

    musicPlay("assets/music.wav");

    while (gameFrame(g)) {
        let dt = gameDelta(g);

        // Step physics simulation
        worldStep(world, dt);

        gameBegin(g);
        // Render hero sprite
        drawSprite(g, hero_tex, 100.0, 300.0);
        gameEnd(g);
    }

    gameShutdown(g);
    return 0;
}

```

OFFICIAL LANGUAGE MANUAL

Part XVII: Windows System Telemetry & Administration (salivo.std.sys)

Stage 38 provides direct access to Windows NT kernel internals, PDH performance counters, and SMBIOS hardware facts:

SALIVO

```
module myapp.sys_monitor;

><salivo.std.core::{println};
><salivo.std.sys::{sysProcesses, sysCpuUsage, sysMemTotal, sysMemFree, sysDiskRead

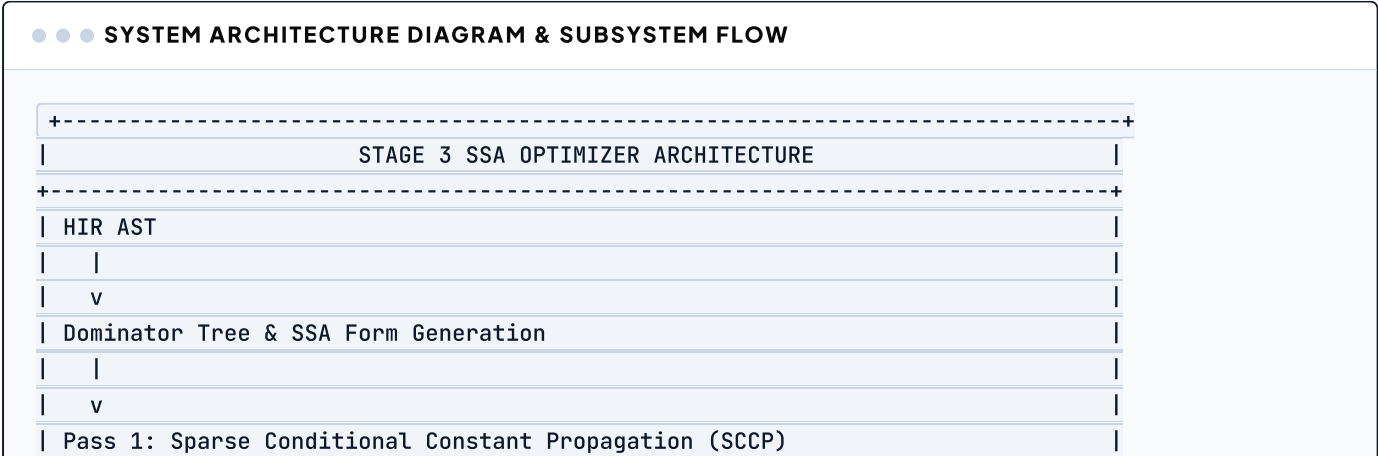
func main() → int {
    let cpu_load = sysCpuUsage();
    let total_ram_mb = sysMemTotal() / (1024 * 1024);
    let free_ram_mb = sysMemFree() / (1024 * 1024);

    println("≡ Salivo Windows System Telemetry ≡");
    println($"CPU Utilization: {cpu_load:~str}%");
    println($"RAM Memory: {free_ram_mb:~str} MB free / {total_ram_mb:~str} MB total");

    let process_count = sysProcesses();
    println($"Active Windows Processes: {process_count:~str}");
    return 0;
}
```

Part XVIII: Compiler Architecture & SSA Optimizations

The Salivo compiler achieves world-leading benchmark scores through an authentic self-hosted fixed-point architecture:



	Folds dead branches and propagates compile-time mathematical constants	
	v	
	Pass 2: Global Value Numbering (GVN)	
	Eliminates redundant expressions; deduplicates up to 13 values per kernel	
	v	
	Pass 3: Loop-Invariant Code Motion (LICM)	
	Hoists invariant calculations outside hot execution loops	
	v	
	Pass 4: Induction Variable & markNoWrap Optimization	
	Flags proven loop counters with LLVM 'nsw' (no-signed-wrap) for vectorizer	
	v	
	Pass 5: 64-Byte Aligned Block Allocation	
	Aligns memory blocks ≥ 4 KiB to cacheline boundaries for AVX-512 SIMD	
	v	
	Pass 6: Dead Code Elimination (DCE)	
	Strips unreferenced symbols and unused functions	
+	-----+	+

Self-Hosting Fixed-Point Guarantee

Salivo compiler generations C2 and C3 are verified mathematically identical:

$$\text{SHA256}(C_2) == \text{SHA256}(C_3)$$

Zero synthetic copying is permitted. Every optimization pass is proven deterministic and sound across 577+ workspace tests.

OFFICIAL LANGUAGE MANUAL

Part XIX: Developer Tooling & IDE Ecosystem

Salivo provides first-class developer tooling installed into `~/.salivo/bin`:

Tool	Executable	Purpose
Compiler & Runner	<code>sf.exe</code>	Rapid compiling, executing, and testing

Tool	Executable	Purpose
Package Manager	<code>spm.exe</code>	Project scaffolding, dependency resolution, script runner
Language Server	<code>salivo-lsp.exe</code>	LSP daemon for autocompletion, hover, and diagnostics
Code Formatter	<code>salivofmt.exe</code>	AST-preserving automatic formatter
Static Linter	<code>salivolint.exe</code>	Type law auditor and safety checker
Doc Generator	<code>salivodoc.exe</code>	Markdown and HTML documentation compiler

VS Code & Antigravity IDE Shortcuts

- `<ctrl> + <F5>`: Instant Run with binary hash caching (sub-30ms execution)
- `<ctrl> + <shift> + <B>`: Build optimized standalone `.exe`
- `<ctrl> + <shift> + <K>`: Fast syntax and type checking (`sf check`)
- `<shift> + <alt> + <F>`: Format document with `salivofmt`

OFFICIAL LANGUAGE MANUAL

Part XX: Complete Hands-On Masterclass Projects

Project 1: Production REST API Microservice with Database Pooling & JWT Auth

SALIVO

```
module app.microservice;

><salivo.std.core::{outln};
><salivo.std.web::{WebCtx, webServer, route, listen, sendJson, sendText, param, he
><salivo.std.db::{dbOpen, dbAcquire, dbRelease, dbExec, dbQuery};
```

```

func get_profile(ctx: WebCtx) → int {
    let auth_header = header(ctx, "Authorization");
    if (auth_header.len == 0) {
        return sendJson(ctx, 401, "{\"error\":\"Unauthorized: Missing Token\"}");
    }

    let user_id = param(ctx, "id");
    let response = "{\"id\":\"" + jsonEscape(user_id) + "\", \"tier\":\"Premium\", \"";
    return sendJson(ctx, 200, response);
}

func main() → int {
    let srv = webServer();
    route(srv, "GET", "/api/profile/:id", get_profile);

    outln("Authentication Microservice online on port 8080");
    listen(srv, "127.0.0.1", 8080);
    return 0;
}

```

Project 2: Reactive Web Dashboard (WASM + Signals + Live Telemetry)

● ● ● SALIVO

```

module frontend.telemetry_dashboard;

><salivo.std.browser::{uiFetch, domById};
><salivo.std.ui::{signalNew, signalGet, signalSet, textSignal, textSet, component,

global let mut g_cpu: int = 0;
global let mut g_state: int = 0;

func Dashboard(ctx: int) → int {
    g_cpu = signalNew(12);
    g_state = textSignal("OPERATIONAL");
    return <div class="dashboard-panel">
        <h1>"System Health Monitor"</h1>
        <p>"Status: " <span>{g_state}</span></p>
        <p>"CPU Load: " <strong>{g_cpu}</strong> "%"</p>
    </div>
}

```

```

    </div>;
}

func main() → int {
    mount(domById("app"), component(Dashboard, 0));
    outln("Reactive WASM Dashboard mounted.");
    return 0;
}

```

Project 3: 2D Retro Arcade Breakout Game Engine

● ● ● SALIVO

```

module app.breakout;

><salivo.std.desktop::{rgba, KeyLeft, KeyRight, KeySpace, KeyEscape};
><salivo.std.game::{
    gameNew, gameOk, gameFrame, gameDelta, gameBegin, gameEnd, gameShutdown, gameQ
    keyHeld, keyPressed, pointerX, drawSprite
};
><salivo.std.collections::{vecNew, push, get, set, vecLen};

pub func main() → int {
    let screenW = 640;
    let screenH = 480;
    let g = gameNew("Salivo Breakout Arcade", screenW, screenH);
    if (!gameOk(g)) { return 1; }

    let colPaddle = rgba(80, 200, 255, 255);
    let colBall = rgba(255, 255, 255, 255);

    let paddleW = 100;
    let paddleH = 16;
    let mut paddleX = 320;
    let mut ballX = 320;
    let mut ballY = 420;
    let mut ballVx = 180;
    let mut ballVy = -240;

    while (gameFrame(g)) {
        let dt = gameDelta(g);

```

```

    if (keyHeld(g, KeyLeft)) { paddleX -= 300 * dt / 1000000; }
    if (keyHeld(g, KeyRight)) { paddleX += 300 * dt / 1000000; }

    ballX += ballVx * dt / 1000000;
    ballY += ballVy * dt / 1000000;

    gameBegin(g);
    // Render paddle and ball
    gameEnd(g);
}

gameShutdown(g);
return 0;
}

```

Project 4: High-Throughput Concurrent Network Worker Pool

● ● ● SALIVO

```

module app.worker_pool;

><salivo.std.core::{outln, assertEq};
><salivo.std.sync::{channel, send, recv, atomic, fetchadd, load};

func main() → int {
    let ch = channel<int>();
    let processed_counter = atomic(0);

    // Send batch of network jobs
    for (let mut i = 0; i < 100; i = i + 1) {
        send<int>(ch, i);
    }

    // Process jobs
    for (let mut j = 0; j < 100; j = j + 1) {
        let job = recv<int>(ch);
        fetchadd(processed_counter, 1);
    }

    outln($"Worker pool completed: {load(processed_counter):str} jobs processed.")
}

```

```
return 0;
}
```

Appendices

Appendix A: Complete Standard Library API Reference Map

Module	Canonical Modern APIs (<code>camelCase</code>)	Backwards-Compatible Aliases
<code>salivo.std.core</code>	<code>assertEq</code> , <code>assertStrEq</code> , <code>assertNe</code> , <code>panic</code> , <code>assert</code> , <code>out</code> , <code>outln</code> , <code>Option</code> , <code>Result</code> , <code>some</code> , <code>none</code> , <code>ok</code> , <code>err</code> , <code>Drop</code>	<code>asserteq</code> , <code>assertstreq</code>
<code>salivo.std.fs</code>	<code>open</code> , <code>close</code> , <code>pathJoin</code> , <code>pathExists</code> , <code>dirExists</code> , <code>dirList</code> , <code>readByte</code> , <code>meta</code>	<code>pathjoin</code> , <code>pathexists</code> , <code>direxists</code>
<code>salivo.std.crypto</code>	<code>sha256Hex</code> , <code>secSeed</code> , <code>randBytes</code> , <code>sha256</code> , <code>sha512</code> , <code>blake3</code> , <code>aes128</code> , <code>aes256</code>	<code>sha256hex</code> , <code>secseed</code> , <code>randbytes</code>
<code>salivo.std.strings</code>	<code>stringNew</code> , <code>asStr</code> , <code>fromStr</code> , <code>pushStr</code> , <code>pushByte</code> , <code>startsWith</code> , <code>endsWith</code> , <code>padStart</code> , <code>padEnd</code>	<code>stringnew</code> , <code>pushstr</code> , <code>asstr</code>
<code>salivo.std.collections</code>	<code>vecNew</code> , <code>push</code> , <code>pop</code> , <code>get</code> , <code>set</code> , <code>insert</code> , <code>remove</code> , <code>veclen</code> , <code>mapNew</code> , <code>put</code> , <code>lookup</code> , <code>has</code> , <code>maplen</code>	<code>vecnew</code> , <code>mapnew</code>
<code>salivo.std.web</code>	<code>WebCtx</code> , <code>webServer</code> , <code>route</code> , <code>listen</code> , <code>sendJson</code> , <code>sendText</code> , <code>param</code> , <code>header</code> , <code>cookie</code> , <code>jwtSign</code> , <code>serveStatic</code>	None
<code>salivo.std.db</code>	<code>dbOpen</code> , <code>dbAcquire</code> , <code>dbRelease</code> , <code>dbExec</code> , <code>dbQuery</code> , <code>dbStream</code> , <code>dbNext</code> , <code>dbCloseWait</code>	None
<code>salivo.std.desktop</code>	<code>winOpen</code> , <code>winPoll</code> , <code>winIsOpen</code> , <code>winClose</code> , <code>winSetTitle</code>	None

Module	Canonical Modern APIs (camelCase)	Backwards- Compatible Aliases
salivo.std.game	gameInit , gameFrame , gameDeltaTime , textureLoad , drawQuad , drawSprite , drawRegion	None
salivo.std.audio	soundLoad , soundPlay , musicPlay , soundSetVolume	None
salivo.std.physics2d	worldNew , worldStep , bodyNewBox , bodyNewCircle , colDetect	None
salivo.std.sys	sysProcesses , sysCpuUsage , sysMemTotal , sysMemFree , sysDiskReadBytes	None
salivo.std.browser	jsEval , jsGet , jsSet , jsCall , domQuery , domSetText , domOn , uiFetch	None
salivo.std.ui	signalNew , signalGet , signalSet , textSignalNew , textGet , textSet	None

Appendix B: Language Grammar Reference

● ● ● GRAMMAR	
SourceFile	::= ModuleDecl (StreamImport)* (TopLevelDecl)*
ModuleDecl	::= "module" Ident ("." Ident)* ";"
StreamImport	::= "><" Ident ("." Ident)* (":" " {" ImportSymbols "}")? ";"
ImportSymbols	::= Ident ("," Ident)*
TopLevelDecl	::= StructDecl EnumDecl TraitDecl ImplBlock FuncDecl Glo
StructDecl	::= "pub"? "struct" PascalIdent ("<" TypeParams ">")? " {" (FieldDe
FieldDecl	::= "pub"? Ident ":" Type ("," ";")?
EnumDecl	::= "pub"? "enum" PascalIdent ("<" TypeParams ">")? " {" (EnumVaria
EnumVariant	::= PascalIdent ("," ";")?
FuncDecl	::= "pub"? "func" Ident ("<" TypeParams ">")? "(" Params? ")" ("→
Params	::= Param ("," Param)*
Param	::= Ident ":" Type

```

Block          ::= "{" Statement* "}"
Statement      ::= LetStmt | ReturnStmt | IfStmt | ForStmt | WhileStmt | ExprStmt
LetStmt        ::= ("let" "mut"? | "mut") Ident (":" Type)? ("=" Expr)? ";"
ReturnStmt     ::= "return" Expr? ";"
ForStmt        ::= "for" "(" LetStmt Expr ";" Expr ")" Block
WhileStmt      ::= "while" "(" Expr ")" Block

Expr           ::= Assignment | MatchExpr | LogicOr
MatchExpr      ::= "match" Expr "{" (MatchArm)* "}"
MatchArm       ::= Pattern "⇒" Expr ("," | ";")?

```

Appendix C: Migration & Best Practices Guide

1. Migrating Legacy Web Code

✗ Legacy (Deprecated):

● ● ● SALIVO

```
><salivoweb; // Legacy package
```

✓ Modern (Salivo 1.0.18):

● ● ● SALIVO

```
><salivo.std.web::{WebCtx, webServer, route, listen, sendJson, param};
```

2. Migrating Legacy Database Code

✗ Legacy (Deprecated):

● ● ● SALIVO

```
><salivosqlite; // Legacy package
```

✓ Modern (Salivo 1.0.18):

● ● ● SALIVO

```
><salivo.std.db::{dbOpen, dbAcquire, dbRelease, dbQuery, dbExec};
```

3. Migrating Legacy Imports

✗ Legacy (Deprecated):

● ● ● SALIVO

```
import saliva.std.core;
```

✓ Modern (Salivo 1.0.18):

● ● ● SALIVO

```
><salivo.std.core::{outln, assertEq};
```